
Tarea de Machine Learning

 **nticmaster**



U N I V E R S I D A D
COMPLUTENSE
M A D R I D

Carlos San Román Cazorla

Universidad Complutense de Madrid

Máster de IA, Big Data y Data Science

Julio 2026

v. 1.0.0

Índice general

1. Enunciado	3
1.1. Normas para la realización del trabajo	3
1.2. Enunciado	3
1.3. Cuestiones a responder	4
2. Análisis Exploratorio de Datos	5
2.1. Tipos de variables y su correcta clasificación	6
2.2. Estadísticos descriptivos y distribución de los datos	6
2.3. Análisis de valores nulos (<i>missing values</i>)	8
2.4. Análisis de Datos Anómalos (<i>outliers</i>)	10
2.5. Relaciones entre variables	12
2.5.1. Relación de variables numéricas con la variable objetivo	13
2.5.2. Relación de variables categóricas con la variable objetivo	13
2.6. Feature Engineering	14
2.6.1. Eliminación de variables	15
2.6.2. Codificación de variables categóricas	15
2.6.3. Estandarización de las variables	15
3. Máquinas de Vector Soporte	16
3.1. SVM Lineal	16
3.2. SVM con kernel RBF (gaussiano)	17
3.3. Comparación de los modelos	18
4. Ensamblado: Bagging y Stacking	20
4.1. Bagging sobre el mejor modelo individual	20
4.2. Stacking	23
4.2.1. Meta-modelo	24
5. Conclusiones	26
5.1. Comparación global de los modelos	26

1 Enunciado

Este trabajo está orientado a predecir una variable binaria a través de diferentes algoritmos de clasificación.

1.1. Normas para la realización del trabajo

- a) El trabajo se hará de forma individual.
- b) La entrega del trabajo consta de dos partes.
 - i) Memoria del trabajo realizado, entregada en pdf a través del Campus Virtual. La tarea se evaluará exclusivamente con el contenido de la memoria, que debe ser autoexplicativa, incluyendo fragmentos del código empleado si fuera necesario. No se admitirán memorias enviadas en formato `.ipynb` o descargadas de Jupyter (con errores, log y similar).
 - ii) Código en Python, para poder reproducir los resultados, pero no será evaluado.
- c) El trabajo deberá estar explicado (no basta con responder a las cuestiones), indicando, si es necesario, el código utilizado en el pdf. El trabajo explicado debe ser reproducible, no basta con exponer los resultados. Se valora la claridad de exposición en el informe y la estructura. Puede contener Anexos de datos y gráficos o no, todo según vuestro criterio. Se establece un límite de 20 páginas.
- d) Se debe usar la matriz de datos proporcionada: `BBDD_ML_TAREA.csv`
- e) Es importante tener en cuenta que cada técnica exige una depuración/preparación de los datos específicos. No es necesario desarrollar este proceso en cada una de las preguntas, pero es imprescindible explicar para cada técnica, qué parte de la depuración/preparación es necesaria.

1.2. Enunciado

Una empresa de servicios de telecomunicaciones desea construir un modelo predictivo que permita anticipar la pérdida o abandono de clientes. Para ello dispone de un conjunto de datos con múltiples características de uso y comportamiento de sus clientes, representadas mediante variables independientes $V1-V20$, y una variable dependiente Y que codifica si un cliente ha abandonado el servicio o no.

El objetivo principal es desarrollar un sistema de clasificación que determine si un cliente está en riesgo de abandono ($Y = 1$) basándose en sus patrones de consumo y servicio.

El conjunto de datos proporcionado contiene las siguientes variables independientes:

$V1-V20$ (atributos descriptivos anonimizados de clientes, correspondientes a métricas de uso de servicio, planes contratados, llamadas, cargos y actividad con servicio de atención), y la variable dependiente Y (riesgo de abandono).

1.3. Cuestiones a responder

1) Análisis y depuración de datos

Analiza y depura la base de datos proporcionada, documentando de forma clara y justificada cada una de las transformaciones y técnicas de *feature engineering* aplicadas. Explica por qué cada proceso es necesario y cómo contribuye a mejorar el rendimiento de los modelos de predicción.

2) Modelado con Máquina de Vectores de Soporte (SVM)

Entrena modelos de SVM con al menos dos kernels distintos y realizando de forma detallada y justificada el proceso de búsqueda de ajuste de hiperparámetros. Representa gráficamente los resultados de esta búsqueda empleando las métricas *accuracy* y AUC como criterios de evaluación. Justifica cada paso metodológico y compara los dos mejores modelos obtenidos, analizando sus diferencias de rendimiento. Escoge de forma razonada un modelo ganador.

3) Ensamblado: Bagging y Stacking

- **Bagging:** Implementa y ajusta un esquema de bagging sobre el mejor modelo individual encontrado en el apartado anterior. Explica en qué consiste bagging, cuál es su objetivo en este problema y los resultados cuantitativos obtenidos al aplicarlo.
- **Stacking:** Explica y entrena al menos un modelo de stacking. Describe con detalle cada clasificador base empleado, el proceso de combinación de sus predicciones y el rendimiento final del modelo de stacking. Justifica las decisiones tomadas en la selección y configuración de los clasificadores base, así como del meta-modelo.

4) Comparación global de modelos

Desarrolla un análisis comparativo exhaustivo entre los modelos y procesos seleccionados en los apartados 2) y 3). Compara su rendimiento según las métricas de evaluación empleadas, discute las ventajas e inconvenientes de cada enfoque y argumenta las decisiones metodológicas adoptadas. Concluye con una evaluación crítica de cuál(es) modelo(s) resultan más adecuados para el problema planteado y por qué.

2 Análisis Exploratorio de Datos

El objetivo de este primer capítulo será realizar el Análisis Exploratorio de Datos (EDA), analizando y depurando la base de datos proporcionada, documentando clara y justificadamente cada una de las transformaciones aplicadas.

Recurriendo a los apuntes, definimos el **Análisis Exploratorio de Datos** como la fase del análisis cuyo objetivo principal es **conocer, comprender y validar los datos** antes de aplicar modelos estadísticos y de Machine Learning.

Comenzamos el estudio con una primera visualización del dataset para hacernos una idea de la estructura, tipos de variables y valores que se presentan. Tras este primer vistazo al dataset, concluimos que está compuesto por 9200 observaciones para las cuales existen 20 variables explicativas y una variable objetivo tal y como indicaba el enunciado del ejercicio. Posteriormente, identificamos la variable objetivo Y , que codifica el abandono del cliente:

- $Y = 1$: El cliente ha abandonado el servicio (*churn*), la cual es la que queremos predecir y detectar.
- $Y = 0$: El cliente permanece.

En este tipo de problemas es habitual encontrar un dataset desbalanceado, lo cual es coherente porque esperamos que los clientes no abandonen nuestra empresa. Para estudiar la distribución, hacemos un recuento de la clase y su proporción:



Figura 2.1: Distribución de la variable objetivo.

En este caso, tenemos una distribución exactamente igual de 0s y 1s, lo cual es extraño y salta a la vista. Asumimos que el dataset ha sido tratado previamente con el objetivo de balancear el conjunto de datos para facilitar el análisis y la obtención de resultados. Sin embargo, haciendo esto antes de la división entre train y test, podemos incurrir en problemas metodológicos graves, pues habría una gran posibilidad de que entrenásemos y validásemos con los mismos datos, un error que desde el punto de vista de negocio podría ser fatal. Tras la evaluación de duplicados, observamos que hay 5661 observaciones duplicados, más de la mitad de la base de datos. Eliminamos esos duplicados y vemos la distribución resultante:



Figura 2.2: Distribución de la variable objetivo.

Tras la eliminación de duplicados vemos cómo realmente sí se cumplen los casos habituales en este tipo de problemas y tenemos la muestra desbalanceada. Ahora, empezamos con el análisis de los datos.

2.1. Tipos de variables y su correcta clasificación

En Python, se infiere directamente el tipo de las variables, sin embargo, hay algunas que, a pesar de su apariencia numérica, realmente tienen significado nominal, luego su clasificación no es correcta. A continuación, veremos la clasificación actual de las variables y mostraremos el tipo en el que estamos interesados para cambiar:

Cuadro 2.1: Propuesta de tipificación y tratamiento de variables.

Variable	Tipo pandas	Tipo conceptual	Tratamiento
V1 (estado)	object	Categorica nominal	Codificación / agrupación
V2 (antigüedad)	int	Númerica discreta	Estandarizar
V3 (cód. área)	int/object	Categorica nominal	One-hot encoding
V4 (n.º teléfono)	object	Identificador único	Eliminar (evita sobreajuste)
V5, V6 (planes)	object	Categoricas binarias	Mapear a 0/1
V7	int	Númerica discreta	Estandarizar
V8-V19	float/int	Numéricas continuas y discretas (minutos/costes)	Estandarizar
V20	int	Númerica discreta	Estandarizar
Y	int	Binaria — TARGET	—

Por tanto, las listas de variables para tratamiento quedarán de la siguiente manera:

- **Identificador:** ['V4']
- **Categoricas nominales:** ['V1', 'V3']
- **Categoricas binarias:** ['V5', 'V6']
- **Numéricas:** ['V2', 'V7', 'V8', 'V9', 'V10', 'V11', 'V12', 'V13', 'V14', 'V15', 'V16', 'V17', 'V18', 'V19', 'V20']

2.2. Estadísticos descriptivos y distribución de los datos

El siguiente paso dentro del Análisis Exploratorio de Datos es estudiar la distribución de cada una de las variables para la detección temprana de *outliers* y el entendimiento completo del problema. Para ello, se

realiza un descriptivo de las variables, estudiando medias, medianas y varianzas junto con la curtosis y la asimetría, entre otras métricas descriptivas. Esto nos da una primera idea de los valores que toman cada una de las variables y cómo se distribuyen. También, nos permite detectar en qué variables hay nulos. Debido a la restricción en la longitud de la entrega, no mostraremos la tabla de resultados obtenida en el código (para verla, simplemente habrá que acudir a la celda 9 del notebook que se entrega junto a esta memoria). A continuación mostraremos una imagen resumen de las distribuciones

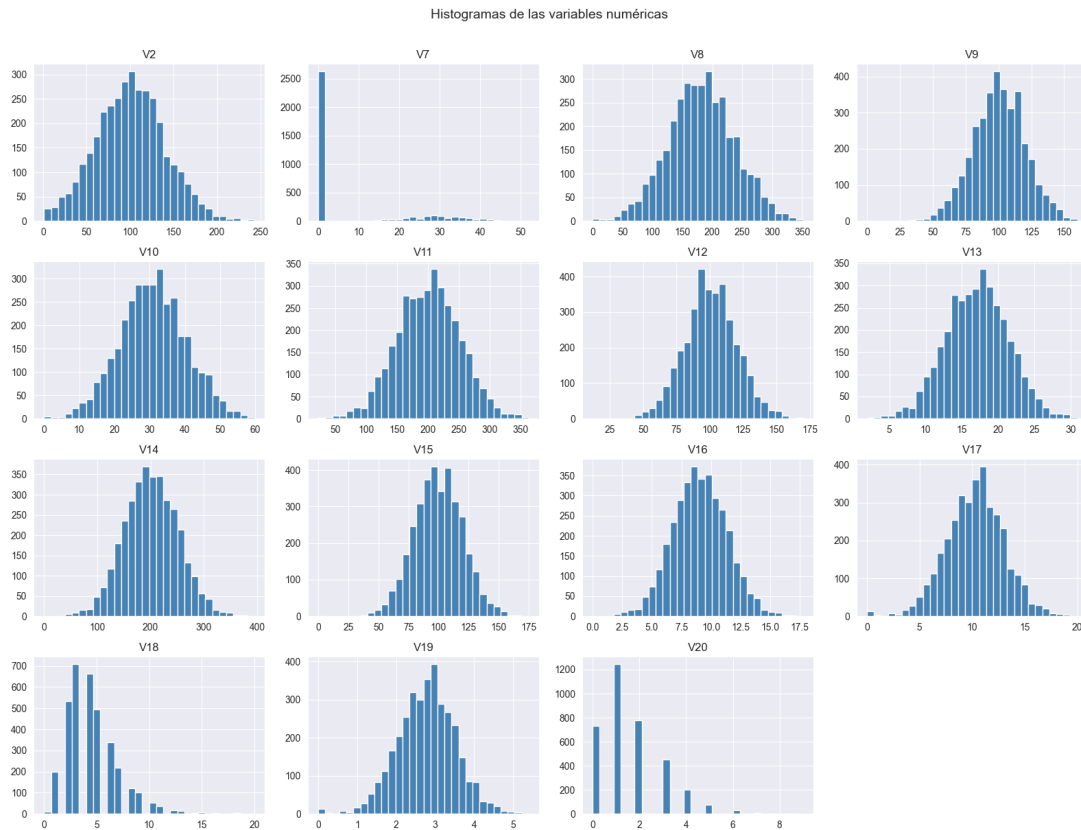


Figura 2.3: Histogramas de las variables numéricas.

Ante los resultados obtenidos, podemos obtener las siguientes conclusiones generales:

- Las variables de **minutos, llamadas y costes (V8-V17)** presentan distribuciones aproximadamente normales, lo cual ayudará desde el punto de vista teórico ya que se cumplirán muchas hipótesis de los modelos.
- Destaca la **V7** puesto que presenta una distribución muy desbalanceada con un peso muy grande en 0. Esto podría parecer un error, sin embargo no es así ya que esta variable mide el número de mensajes de buzón de voz de los clientes, luego es coherente que muchos clientes no reciban o manden estos mensajes, por lo que tiene sentido que prácticamente la totalidad del peso esté en 0.
- Vemos que las variables **V18 y V20** presentan asimetría positiva y colas largas, luego pueden ser candidatas a tener *outliers*, y puede darnos características distintivas para la clasificación.
- Finalmente, vemos que muchas de las variables tienen la misma distribución, con ligeras variaciones, lo que nos puede avisar de que minutos de llamada y precio de las llamadas presentan una relación prácticamente lineal.

Por otro lado, también se hace un descriptivo de las variables categóricas, haciendo un recuento de las categorías y del número de observaciones pertenecientes a cada categoría. Ante esto, obtenemos los siguientes resultados:

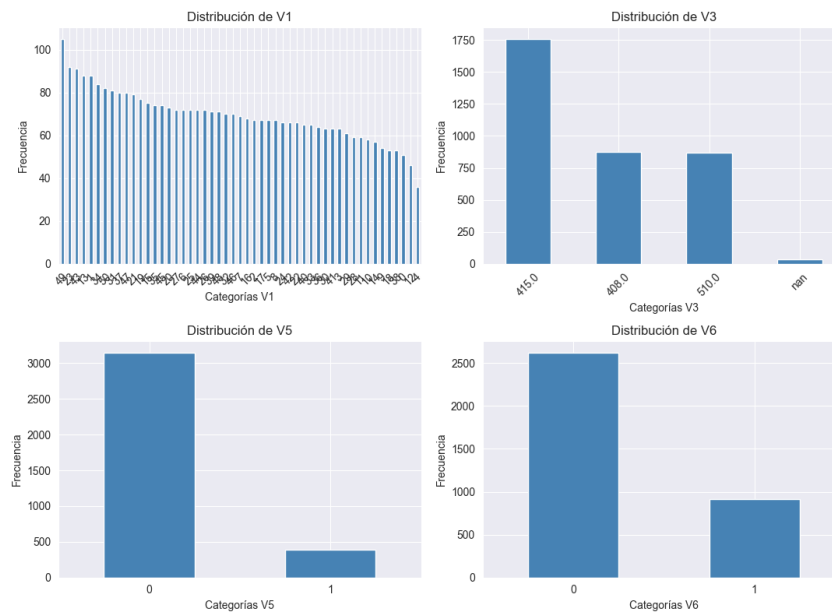


Figura 2.4: Frecuencias variables categóricas.

Como primer análisis, vemos que las variables V5 y V6 pueden tener un alto valor predictivo ya que hay desbalanceo entre ellas.

2.3. Análisis de valores nulos (*missing values*)

El siguiente paso dentro de este capítulo es la detección de valores nulos. Previamente, con el descriptivo del dataset hemos podido visualizar las variables que contenían *missings*. A continuación, debemos de tener en cuenta que puede que no todos los *missings* estén codificados de la misma manera. En el análisis exploratorio del dataset no hemos detectados valores extraños que nos indicasen *missings* codificados de forma no estándar, luego buscamos codificaciones habituales como strings vacíos, ?, NAs, espacios o valores imposibles (como valores negativos en variables de conteo, minutos o costes). Frente a estas hipótesis, no detectamos ninguna de estas codificaciones, por lo que continuamos con el estudio. Ahora, presentamos el número de *missings* y sus correspondientes porcentajes en el global de las variables:

Salida de Consola: Valores Nulos

	n_missing	pct_missing
V14	28	0.79
V15	26	0.73
V19	16	0.45
V10	13	0.37

Observamos que tienen un porcentaje muy pequeño de missings, prácticamente insignificantes en cada variable. Representan menos de un 3% de las observaciones. A continuación, procederemos a ver cómo

se tratan estos missings. Para tratar los *missings* es importante tener en cuenta el tipo de valor perdido. Definimos estos tipos de la siguiente manera:

- **MCAR (Missing Completely at Random):** La probabilidad de que un valor sea *missing* no depende del(los) valor(es) de la(s) variable(s) objetivo que se imputarán ni de los valores de las variables auxiliares.
- **MAR (Missing At Random):** La probabilidad de que un valor sea *missing* depende de un valor de las variables auxiliares, pero no depende del(los) valor(es) de la(s) variable(s) objetivo que se imputarán.
- **NMAR (Not Missing At Random):** La probabilidad de que un valor sea *missing* depende del(los) valor(es) de la(s) variable(s) objetivo que se imputarán y de los valores de las variables auxiliares.

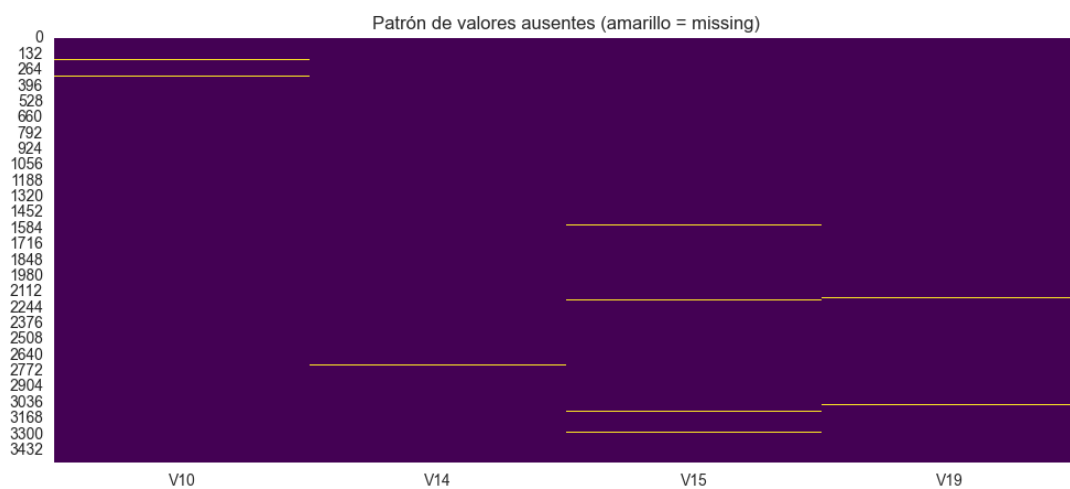


Figura 2.5: Patrón de *missings*.

La figura 2.5 nos sirve para estudiar el patrón de valores faltantes y sus relaciones tanto con la variable objetivo como con el resto de variables. Para identificar el tipo de *missing*, empleamos el test de Little (si la distribución de *missings* es aleatoria). El objetivo es determinar si la probabilidad de *missing* depende de la variable objetivo. Tras la realización de los tests, obtenemos los siguientes resultados:

Clasificación de Valores Nulos

V10: χ^2 vs Y $\rightarrow$ p-valor = 0.4468 (compatible con MCAR)
V14: χ^2 vs Y $\rightarrow$ p-valor = 0.8780 (compatible con MCAR)
V15: χ^2 vs Y $\rightarrow$ p-valor = 0.6057 (compatible con MCAR)
V19: χ^2 vs Y $\rightarrow$ p-valor = 0.8473 (compatible con MCAR)

Por tanto, ante los resultados obtenidos tendremos que decidir cómo imputar estos datos. Según el objetivo del problema, podemos hacer distintos tipos de imputación. Si queremos hacer un análisis descriptivo o realizar un clustering, podemos imputar por métricas como la media o la mediana, puesto que esto no perturba la distribución (Esto lo haremos para los casos MCAR).

Por otro lado, si nuestro objetivo es predecir, esta no es la mejor de las opciones ya que estaríamos desvirtuando la distribución y estaríamos dando más peso a esos valores. Además, en casos de datos perdidos MAR y NMAR, es justo lo que no podemos hacer puesto que están perdidos por ser valores

que se distinguían de los valores medios. Luego para tratar de predecirlos, haríamos un nuevo problema donde la variable objetivo sería la variable perdida y las explicativas serían el resto, por lo que haríamos una predicción de ese valor perdido. En este caso, como todas las variables son compatibles con MCAR y son pocas observaciones, imputaremos por la mediana ya que no desvirtuará la muestra.

Así, no quedan datos ausentes por lo que podemos pasar al siguiente paso del EDA: Análisis de Datos Anómalos.

2.4. Análisis de Datos Anómalos (*outliers*)

A continuación, nos enfrentamos a una parte crucial dentro del EDA. Debemos ser capaces de detectar los datos anómalos y estudiar la corrección de estos datos. Para detectarlos y ver su distribución en cada una de las variables, primero procedemos con una visualización gráfica mediante *boxplots*:

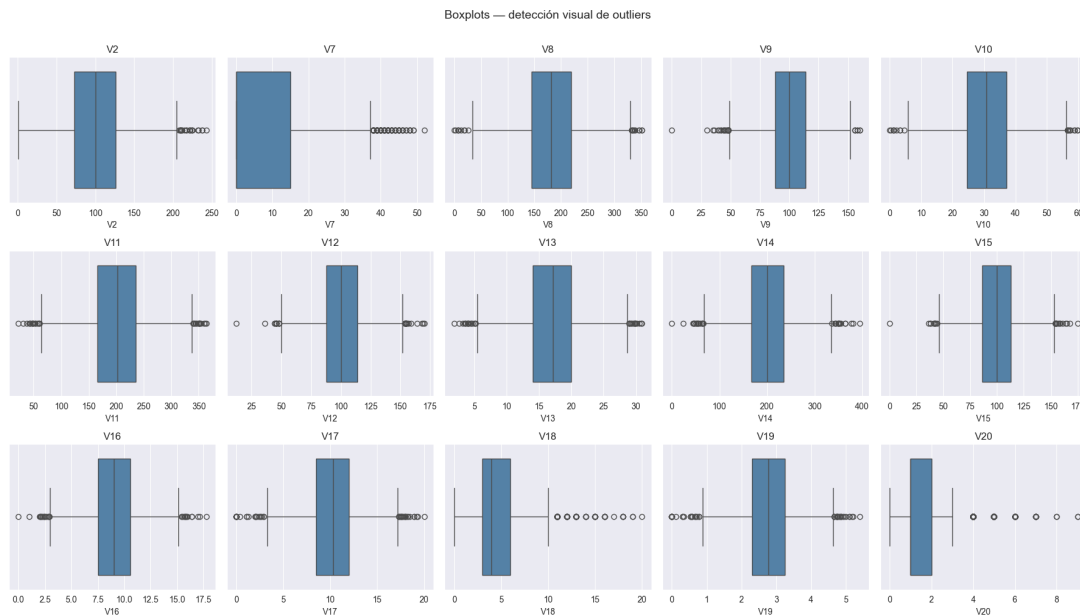


Figura 2.6: Boxplots de variables.

Atendiendo a los resultados obtenidos, podemos ver características de los clientes de la empresa. Por ejemplo, para V2 existen datos anómalos a la derecha de la distribución, indicando que llevan mucho tiempo con la empresa, luego podemos inferir que son clientes que están satisfechos con el servicio y no buscan un cambio. Por otro lado, para V7, vemos que clasifica prácticamente todos los valores distintos de 0 como *outliers* puesto que era una distribución muy desbalanceada y ese valor concentraba todo el peso de la distribución.

Una situación que puede parecer peculiar es la distribución de los *outliers* en las variables V18 y V20, puesto que son valores equidistantes que siguen todos el mismo patrón. Sin embargo, en el resto de las variables vemos que los atípicos suelen distribuirse más como colas alargadas, es decir, con más peso cerca de los cuartiles y se van difuminando según se van alejando de los bigotes.

A pesar de este análisis visual, no podemos decidir el tratamiento de estos valores atípicos simplemente por una herramienta gráfica, por lo que recurrimos a métodos analíticos como la regla del rango intercuartílico (IQR) y el *z-score*. Estos son los resultados obtenidos:

Variable	N.º Outliers (IQR)	% Outliers (IQR)	N.º Outliers ($Z > 3$)
V2	22	0.62	11
V7	150	4.24	7
V8	19	0.54	8
V9	27	0.76	8
V10	19	0.54	8
V11	31	0.88	12
V12	29	0.82	6
V13	31	0.88	12
V14	32	0.90	14
V15	24	0.68	9
V16	31	0.88	12
V18	93	2.63	23
V19	49	1.38	24
V20	326	9.21	45

Cuadro 2.2: Análisis de valores atípicos (outliers) mediante los criterios IQR y Z-score.

También, miremos el impacto de los *outliers* sobre la media y la desviación típica para hacer un correcto tratamiento:

Variable	Media		Desviación Estándar	
	Con Outliers	Sin Outliers	Con Outliers	Sin Outliers
V2	100.676	100.275	39.697	39.101
V7	7.587	7.506	13.454	13.341
V8	181.799	182.002	55.501	54.941
V9	100.227	100.312	19.778	19.527
V10	30.906	30.941	9.435	9.340
V11	200.772	200.958	51.026	50.244
V12	100.578	100.545	19.615	19.414
V13	17.066	17.082	4.337	4.271
V14	200.779	200.684	50.408	49.391
V15	99.677	99.664	20.000	19.711
V16	9.039	9.038	2.276	2.235
V17	10.264	10.302	2.747	2.642
V18	4.443	4.283	2.506	2.179
V19	2.771	2.781	0.741	0.711
V20	1.599	1.537	1.355	1.243

Cuadro 2.3: Comparativa extendida de estadísticos descriptivos pre y post tratamiento de outliers.

Como se puede ver en esta simulación sin *outliers*, los atípicos no tienen prácticamente peso sobre estas métricas, por lo que nos hace indicar que son posibles casos de negocio y por tanto, se mantienen para que el modelo capte su efecto.

Muchas veces los *outliers* se eliminan porque dan problemas que el modelo no puede estimar, sin embargo, muchas veces esto está mal hecho ya que, en problemas como estos, son justo esos casos los que dan puntos clave para detectar el abandono. Por ello, es muy importante tener a un experto en datos que confirme la corrección de estos datos. En nuestro problema concreto, los valores extremos detectados (clientes con muchos minutos, muchas llamadas al servicio de atención,...) son valores reales del negocio, no errores de medición o transcripción: un cliente puede llamar perfectamente 9 veces a atención al cliente, y de hecho, esos son muchas veces los casos más informativos para predecir el abandono (ya sea porque está satisfecho con el servicio recibido y por tanto, se quede; o bien, no le guste el trato y no le resuelven el problema, por lo que se va de la empresa). Por tanto, llegamos a las siguientes conclusiones de cara a la modelización posterior:

1. **No se eliminan las filas:** podríamos perder clientes muy riesgosos (además, eliminar el 2-5 % de filas por variable acumularía una pérdida importante).

2. Solo se corregirían (imputando como *missing*) valores **imposibles** (negativos, minutos >24h/día), detectados en el apartado anterior.
3. El impacto de los outliers sobre el SVM se **mitiga con la estandarización** (StandardScaler) y con el margen blando (parámetro C): el SVM con regularización adecuada es razonablemente robusto.

2.5. Relaciones entre variables

En esta última sección del capítulo, analizaremos las relaciones entre las variables por si existiese algo de redundancia o colinealidad y la relación que tienen las variables con la variable objetivo Y .

La primera métrica que estudiamos es la correlación, la cual mide la relación lineal existente entre variables y para ello, recurriremos a la matriz de correlaciones, como se muestra a continuación:

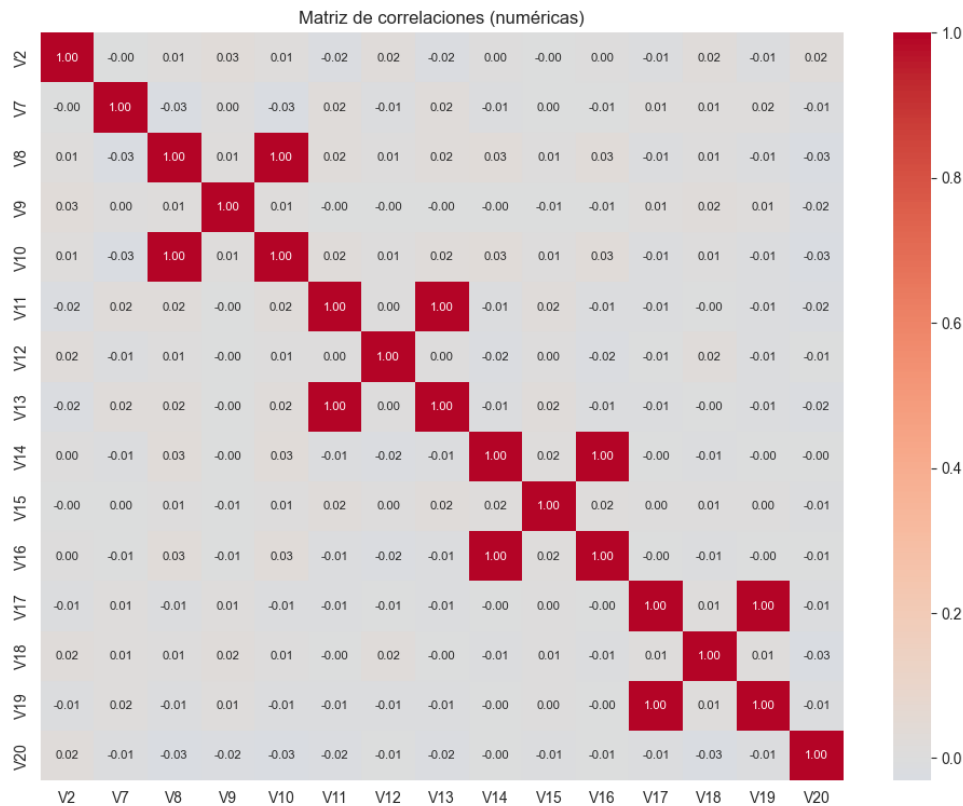


Figura 2.7: Matriz de correlaciones.

Como se puede apreciar claramente, existen variables cuya correlación es exactamente 1. Esto significa que la relación entre las variables es lineal, luego aportan exactamente la misma información. Esto ocurre entre las variables minutos y costes de cada franja horaria (V8–V10, V11–V13, V14–V16, V17–V19), puesto que la función coste es una función lineal $coste = tarifa \cdot minuto$, luego obtenemos redundancia.

Ante esto, para cada par, eliminaremos los costes y conservaremos los minutos. Esta decisión se justifica puesto que la multicolinealidad perfecta no aporta información nueva y añade dimensiones innecesarias. Además, como el modelo que debemos hacer es un SVM con distintos kernels, seleccionaremos el kernel lineal, al cual le viene bien para mejorar la estabilidad e interpretabilidad; y en el caso del kernel gaussiano, reducir estas dimensiones nos permitirá reducir el coste computacional.

2.5.1. Relación de variables numéricas con la variable objetivo

Una parte muy importante de las relaciones entre variables es la relación con la variable objetivo. Esto nos sirve para detectar si existe alguna de las características que divida perfectamente el espacio y por tanto, nos permita predecir la variable objetivo. Para hacerlo, hacemos una visualización de la distribución de cada una de las variables dividida según la variable objetivo. Haciendo esto, obtenemos los siguientes resultados:

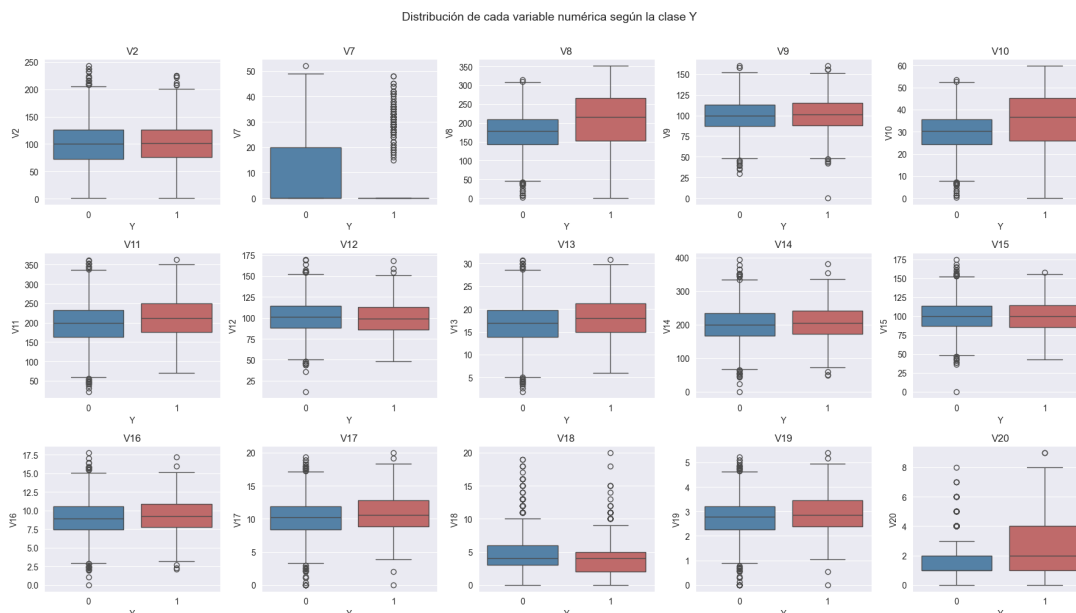


Figura 2.8: Relación de variables numéricas con la target.

Como vemos, ante estas distribuciones y relaciones con la variable objetivo, no conseguimos detectar ninguna variable que determine con seguridad si un cliente se va a dar de baja o no, puesto que todos los diagramas están muy nivelados. Al estar las dos decisiones bastante igualadas en todas las variables, no tenemos una característica con fuerte carácter predictivo, luego no podemos tomar una decisión basándonos en una variable. Tal vez V7 podría darnos algo más de poder predictivo, pero al ser una variable tan desbalanceada, resulta complicado asegurarlo. De hecho, con el test de Mann-Whitney llegamos a que no existe evidencia significativa para que alguna clasifique la variable objetivo.

2.5.2. Relación de variables categóricas con la variable objetivo

Una vez hemos obtenido las relaciones con las numéricas, también es importante estudiar las relaciones con las categóricas. Pudiera ocurrir que existiese alguna categoría para la cual tuviésemos la certeza de que si una observación tuviese ese valor, entonces la podríamos clasificar en alguna de las clases de la variable objetivo. De nuevo, hacemos una visualización de las tasas por cada variable categórica:

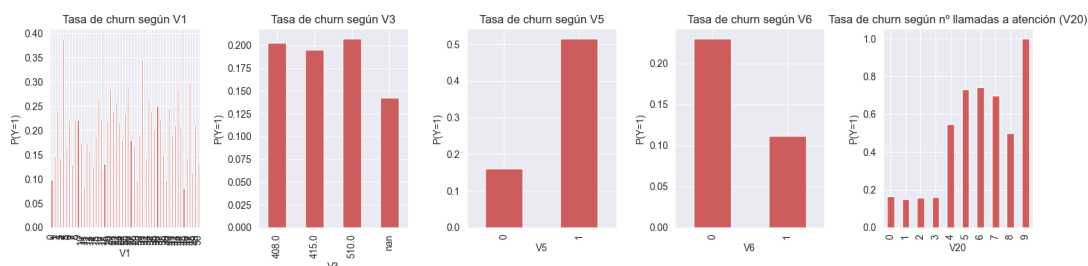


Figura 2.9: Relación de variables categóricas con la target.

Hagamos una revisión de los resultados que se pueden obtener de los gráficos:

- **V1:** En este caso, tenemos muchas categorías por lo que es difícil generalizar un resultado. Mediante el estudio de la tasa de abandono con esta variable tratábamos de estudiar si existía algún estado o provincia para la cual la tasa de abandono fuese elevada y por tanto, nos ayudase a clasificar y explicar la variable objetivo. Sin embargo, no existe un corte claro que nos permita clasificar a la perfección el abandono.
- **V3:** Para los tres valores, la tasa de abandono está igualada por lo que no nos permite clasificar si para alguno de estos valores existe abandono, luego no es una variable explicativa en términos de clasificación.
- **V5:** En este caso, vemos cómo si se tiene el plan internacional, los clientes tienden a abandonar mucho más la empresa. Esta variable es muy predictiva y además explicativa. Como empresa deberíamos replantearnos si no tenemos un buen plan internacional puesto que muchos de los que lo contratan tienden a abandonarlo.
- **V6:** Al igual que en el caso anterior, nos encontramos con una variable más explicativa, sin embargo es en menor medida, puesto que la tasa es mucho menor. En este caso, podemos ver que si tenemos el plan de buzón de voz contratado, es más probable quedarse. Luego, como decisión de negocio, tal vez podríamos plantearnos ofrecerlo más para afianzar a nuestros clientes.
- **V20:** Aunque esta variable es numérica (pero discreta), también merece la pena un tratamiento especial, puesto que como vemos, existe un número de llamadas al servicio de atención a partir del cual la tasa de abandono crece a gran escala. Como vemos, a partir de las 4 llamadas, la tasa de abandono crece diferenciándose de las anteriores, luego eso podría implicar que nuestros clientes no están satisfechos con nuestra atención y por ende, no les solucionamos los problemas.

Para tener un mayor detalle de estas tasas, se recomienda acudir al código donde viene con mayor desglose debido a la restricción en la longitud de la memoria.

2.6. Feature Engineering

Antes de entrenar la máquina de vectores soporte (SVM) es necesario adaptar el conjunto de datos a los requisitos del algoritmo, que solo admite entradas numéricas y es especialmente sensible a la escala y a la dimensionalidad de las variables. A continuación se justifica cada una de las transformaciones aplicadas.

2.6.1. Eliminación de variables

En primer lugar, se elimina la variable **V4** (número de teléfono), ya que se trata de un identificador único de cada cliente. No generaliza: cada valor aparece una sola vez en la muestra, luego cualquier patrón que el modelo pudiera extraer de ella sería puro sobreajuste.

En segundo lugar, se descartan las variables de coste **V10**, **V13**, **V16** y **V19**. Estas variables presentan una correlación prácticamente perfecta con sus correspondientes variables de minutos. Mantener ambas versiones no añadiría información e introduciría colinealidad y dimensiones innecesarias en el espacio de características.

Por último, se elimina la variable **V1** (estado de residencia). Aunque podría contener información geográfica relevante, tiene una cardinalidad muy elevada con frecuencias bajas por categoría, de modo que una codificación *one-hot* generaría 50 variables *dummy*. Esto perjudicaría la dimensionalidad, especialmente para un método basado en distancias como la SVM con kernel RBF. Además, el EDA muestra una relación débil con la variable objetivo, por lo que el coste dimensional no queda compensado por la ganancia predictiva.

2.6.2. Codificación de variables categóricas

Dado que la SVM opera únicamente sobre vectores numéricos, las variables categóricas restantes deben codificarse. Las variables binarias **V5** y **V6** (plan internacional y plan de buzón de voz) se transforman directamente a valores 0/1, lo que preserva toda su información sin aumentar la dimensión del problema.

La variable **V3** (código de área) es categórica nominal de baja cardinalidad, con únicamente tres códigos distintos. En este caso la codificación *one-hot* es la opción natural: genera solo tres variables *dummy*, evita imponer un orden artificial entre categorías (como haría una codificación ordinal) y no plantea ningún problema de dimensionalidad.

2.6.3. Estandarización de las variables

Finalmente, se aplica una estandarización de todas las variables numéricas mediante **StandardScaler**, transformando cada variable para que tenga media cero y desviación típica unitaria:

$$z = \frac{x - \mu}{\sigma}. \quad (2.1)$$

Este paso es **imprescindible** en una SVM. El algoritmo se fundamenta en el cálculo de distancias y productos escalares entre observaciones, por lo que las variables de mayor magnitud dominarían el resultado.

Es fundamental subrayar que los parámetros de la estandarización (μ y σ de cada variable) se ajustan **exclusivamente con el conjunto de entrenamiento** y se aplican después, ya fijados, al conjunto de test.

3 Máquinas de Vector Soporte

El presente capítulo tiene por objeto el desarrollo de varios modelos de SVM con distintos *kernels* para tratar de realizar predicciones y clasificaciones a partir del conjunto de datos tratado en el capítulo anterior.

Se pide entrenar modelos de SVM realizando de forma detallada y justificada el proceso de ajuste de hiperparámetros. Como vimos en teoría, el objetivo de los SVM es plantear un problema de separación lineal de clases mediante métodos algebraicos. Este proceso se puede generalizar a una separación no lineal cuando los datos no sean linealmente separables. Por ello, nosotros entrenaremos dos SVMs con los siguientes *kernels*:

1. **Kernel lineal:** Se supone que existe un hiperplano capaz de separar las clases en el espacio original. Puesto que esto no suele existir, es necesario permitir observaciones mal clasificadas para no incurrir en sobreajuste. Por ello, existe una constante C de regularización que ajustamos para permitir una mayor tasa de fallo (A mayor C , menor margen).
2. **Kernel gaussiano:** Al ser muchas veces la separación entre clases no lineal, una idea para conseguirlo consiste en proyectar el conjunto de datos en un espacio de dimensión superior donde sí tenga sentido la separación. En este caso particular, se realiza una transformación no lineal de los datos en el espacio original al mapearlos a un espacio de características de dimensionalidad infinita. El kernel utilizado presenta la siguiente forma:

$$K(x, z) = \exp\left(-\frac{\|x - z\|^2}{2\sigma^2}\right)$$

donde x y z son los vectores de características originales, $\|\cdot\|$ representa la norma euclídea, y σ es un parámetro que controla la anchura del kernel.

3.1. SVM Lineal

En esta sección debemos desarrollar un modelo de SVM lineal que trate de resolver el problema de abandono. Para ello, productivizaremos el modelo para realizar una búsqueda paramétrica en rejilla.

El primer paso que debemos realizar para construir el SVM lineal es la estandarización de los datos. Para ello, utilizaremos la función de *sklearn* `StandardScaler()`, la cual realiza la transformación $z = \frac{x - \mu}{\sigma}$ donde x es la característica, μ es la media de la característica y σ su desviación típica. Como puntualización importante, recordar que los parámetros μ y σ son del conjunto de entrenamiento y se utilizarán esos mismos para el conjunto de test en la validación.

Como se ha mencionado en el apartado anterior, debemos ajustar un parámetro C que mide la regularización. Para tratar de obtener el mejor modelo, se hará una búsqueda en rejilla, lo cual nos permite evitar la búsqueda manual, a pesar de que tenga un coste computacional más alto. Posteriormente, se realizará una validación cruzada para todos los modelo buscando aquel con mayor robustez y métricas. De hecho, no podemos pasar por alto dos ajustes importante: al ajustar el parámetro C , no dejamos el mismo peso para las dos clases. Al ser un problema desbalanceado, la SVM castiga mucho más los errores cometidos sobre los 1s durante la optimización del hiperplano. Por otro lado, elegimos maximizar el *área bajo la curva ROC* en lugar del *accuracy* ya que un modelo “vago” clasificaría todo como 0s y obtendría un 80 % de *accuracy*. De esta manera, aplicamos técnicas de rebalanceo sin modificar el conjunto de entrenamiento, creand duplicados, lo que podría llevarnos a un sobreajuste y convertir nuestro clasificador en demasiado sofisticado y tener *falsos negativos*.

Realizando todo el proceso, llegamos a los siguientes resultados:

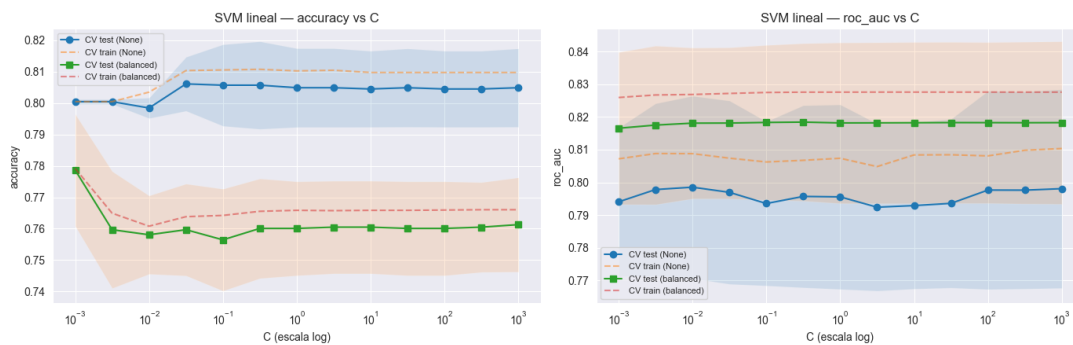


Figura 3.1: Resultados kernel lineal tras CV.

Hemos representado una figura en la que vemos la amplitud de la métrica en cada uno de los folds y como figura central la métrica media para los conjuntos *train* y *test*. Tal y como esperábamos, si no hubiésemos aplicado esa opción de rebalanceo, en cuanto a *accuracy*, el modelo se comportaba mucho mejor que utilizando la versión de balanceado (puesto que clasificaba todo mayoritariamente como 0). Sin embargo, a la hora de evaluar el *AUC-ROC*, la eficiencia caía y el modelo que daba mayor peso a la clase del 1 se comportaba mucho mejor y era mucho más robusto. En términos generales, no se ha obtenido un mal modelo puesot que las métricas están en torno al 0,80, pero teniendo en cuenta el desbalanceo de clases en torno a este valor, no podemos asegurar que sea el clasificador perfecto. Obtenemos un modelo final de 0,7601 de *accuracy medio* y de *AUC medio* = 0,0,8184. Veamos si empleando el truco del kernel podemos mejorar el modelo.

3.2. SVM con kernel RBF (gaussiano)

Previamente ya hemos presentado el *background teórico* en el que se sustenta este tipo de modelo. La metodología a seguir es la misma que anteriormente: haremos una estandarización de los datos, puesto que al tener distintas unidades, aquellas de mayor magnitud serían las de mayor peso dentro del modelo, luego podríamos incurrir a error si no fuesen las de mayor poder predictivo. Tras realizar esta primera estandarización (siempre con los parámetros del conjunto de *train*), haremos una búsqueda en rejilla de los parámetros necesarios para la regularización de este modelo, también con una búsqueda balanceada y tratando de optimizar el *AUC-ROC*.

De nuevo, en este tipo de modelo aparece el parámetro C , donde su aumento implica menor sesgo y mayor sobreajuste. Por otro lado, aparece un nuevo parámetro $\gamma = -\frac{1}{2\sigma^2}$, donde recordemos que σ era un parámetro que controlaba la anchura del kernel. Volvemos a aplicar una validación cruzada para todos los modelos surgidos de la búsqueda en rejilla. Tras el entrenamiento, obtenemos los siguientes resultados:

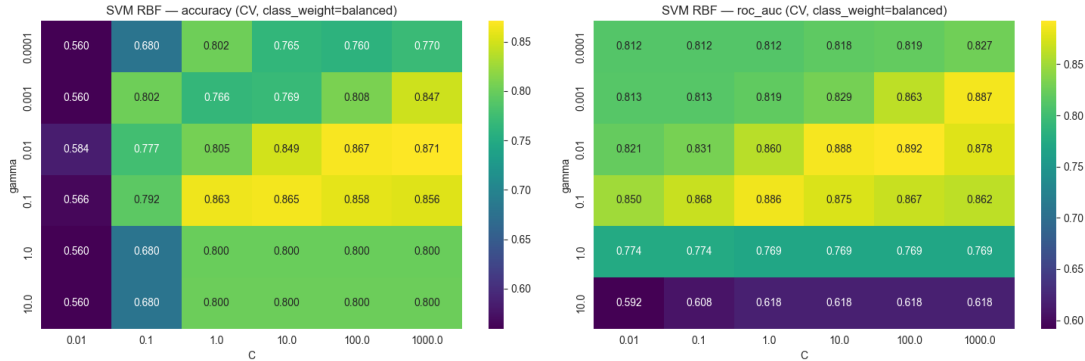


Figura 3.2: Resultados kernel gaussiano tras CV.

Tras los resultados obtenidos, vemos una buena mejora de métricas: el truco del kernel funciona. Tras la búsqueda en rejilla, vemos que hay una diagonal que determina los buenos modelos. En una primera selección, tomamos $C = 100, \gamma = 0,01$, como modelo equilibrado con *accuracy medio* = 0,8671 y *AUC medio* = 0,8922.

Ante los buenos resultados obtenidos, queremos ver si podemos mejorar aún más las métricas. Por ello, nos centramos en los parámetros óptimos del anterior modelo y ampliamos la rejilla, buscando parámetros en un entorno suyo. Reiterando el proceso, obtenemos un mejor resultado con la rejilla fina para los parámetros $C = 50,1187, \gamma = 0,01$ y obteniendo un *AUC promedio* = 0,8923.

Observación: La diagonal que mencionábamos anteriormente presenta la interdependencia existente entre los parámetros C y γ , es decir, valores bajos de γ compensan un C alto y viceversa (vemos la gran diferencia en magnitud entre esos valores con los valores obtenidos). La interpretación del sentido geométrico del valor de γ es que valores demasiado altos de γ colapsan el rendimiento, llegando al sobreajuste puesto que estrechan mucho el kernel. Por otro lado, valores demasiado bajos lo aplanan hacia un comportamiento lineal.

3.3. Comparación de los modelos

Para seleccionar el modelo ganador, recurriremos al conjunto de test. Ante los resultado obtenidos anteriormente, aunque el SVM lineal presenta una buena *performance*, el SVM gaussiano generaliza y logra clasificar prácticamente a la perfección todos los casos. Para verlo, recurrimos a la tabla de métricas, las matrices de confusión y la curva ROC:

Cuadro 3.1: Comparativa de rendimiento en Test: SVM Lineal vs. SVM RBF

Métrica / Clase	SVM Lineal			SVM RBF		
	Prec.	Rec.	F1	Prec.	Rec.	F1
Clase 0	0.941	0.748	0.834	0.959	0.872	0.913
Clase 1	0.446	0.811	0.575	0.623	0.849	0.719
Macro Avg	0.693	0.780	0.704	0.791	0.860	0.816
Weighted Avg	0.842	0.761	0.782	0.892	0.867	0.874
Accuracy (Test)	0.7608			0.8672		
AUC (Test)	0.8493			0.9211		
AUC (CV)	0.8184			0.8923		

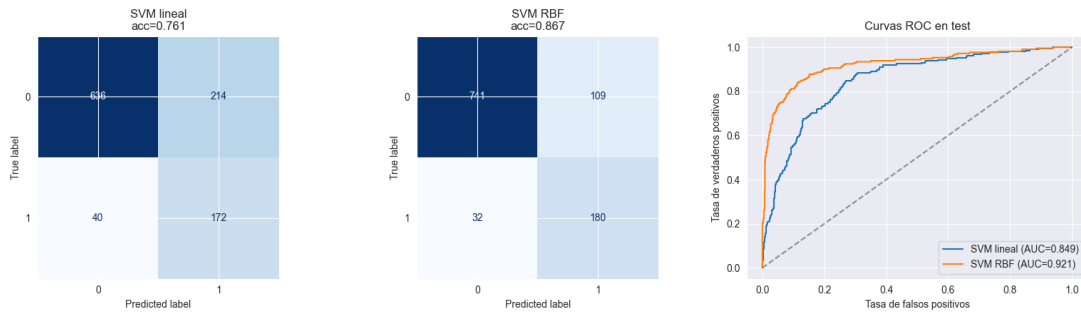


Figura 3.3: Resultados SVM.

Como vemos, el SVM gaussiano mejora al kernel lineal obteniendo mejores métricas. En ambos casos, las SVM fallan en mayor medida clasificando 0s como 1s, es decir, predicen más falsos positivos. Desde un punto de vista de negocio esto es mucho mejor: prefiero destinar dinero a campañas de retención aunque haya clientes que no se vayan a ir frente a que se me vayan clientes que no tenía previsto. Como conclusiones globales podemos enumerar las siguientes:

1. **Rendimiento predictivo y capacidad no lineal:** El **SVM RBF** supera al modelo lineal (en *Accuracy* y AUC). Esta mejora responde a las relaciones no lineales identificadas en el EDA, las cuales requieren la proyección a un espacio de mayor dimensión para trazar una frontera de decisión adecuada.
2. **Consistencia CV–Test:** Muestra un comportamiento coherente entre la validación cruzada y el conjunto de test, garantizando un buen equilibrio entre sesgo y varianza y descartando sobreajuste en la búsqueda de hiperparámetros.
3. **Gestión del coste de los errores (*Recall*):** En un contexto de *churn*, un falso negativo (cliente perdido no identificado) implica un coste mayor que un falso positivo. El SVM RBF maximiza el *recall* de la clase positiva (clase 1), optimizando la capacidad de detección.
4. **Compromiso explicabilidad–precisión:** Aunque el kernel lineal ofrece mayor explicabilidad y menor coste computacional, la naturaleza del problema de negocio justifica asumir esta contrapartida en favor de un mayor rendimiento predictivo.

Por tanto, se selecciona el **SVM gaussiano como modelo ganador**, constituyéndose como el modelo base para los *ensamblados* del Capítulo 4.

4 Ensamblado: Bagging y Stacking

En el campo del aprendizaje automático, los **métodos de ensamblado** utilizan múltiples algoritmos para obtener un rendimiento predictivo que mejore el que podría obtenerse por medio de cualquiera de los algoritmos de aprendizaje individuales que lo constituyen. Es decir, en lugar de optimizar un único modelo, se combinan *varios modelos* para obtener un resultado más robusto y preciso. En este capítulo aplicaremos los métodos de **Bagging** y de **Stacking** para tratar de mejorar las métricas del modelo obtenido en el capítulo anterior.

Es importante recordar que los métodos de ensamblado heredan los requisitos de sus modelos base. Como en el método de Stacking usamos modelos basados en distancias como el KNN o modelos sensibles a las unidades como la regresión logística, utilizaremos la matriz de datos estandarizados obtenida en la depuración. En el caso de los modelos de árboles, esta depuración no es necesaria pero tampoco le afecta por lo que también podremos utilizarla. Finalmente, es importante tener en cuenta que al combinar los resultados obtenidos de los distintos modelos, debemos tener las mismas escalas en las predicciones, por lo que combinaremos **probabilidades** y no salidas de distinta naturaleza.

4.1. Bagging sobre el mejor modelo individual

La agregación *bootstrap* (*Bootstrap Aggregating* o *bagging*) es un meta-algoritmo que construye una familia de modelos a partir de un mismo algoritmo de aprendizaje, con el objetivo de **disminuir la varianza** del error y, con ello, mitigar el sobreajuste. El procedimiento para construir B modelos consta de cuatro pasos:

1. **Remuestreo:** se extraen B muestras aleatorias simples **con reemplazamiento** del conjunto de entrenamiento.
2. **Entrenamiento:** se ajusta un modelo individual sobre cada una de esas muestras.
3. **Predicción:** cada uno de los B modelos emite su predicción sobre las observaciones nuevas.
4. **Agregación:** se combinan las B predicciones (voto mayoritario, promediando probabilidades estimadas...)

El modelo ganador del capítulo anterior es un SVM con kernel gaussiano: presenta un sesgo bajo, pero a cambio su frontera de decisión depende de un número reducido de observaciones concretas (los vectores soporte). Esto lo convierte en un modelo potencialmente **sensible a la muestra de entrenamiento**: pequeñas variaciones en los datos podrían desplazar la frontera. El bagging ataca precisamente esa componente del error.

En el cuadro 4.1 se recogen las decisiones tomadas en la configuración del ensamblado junto con su justificación.

Decisión	Valor	Justificación
Estimador base	<i>Pipeline</i> escalado + SVC(RBF) con los hiperparámetros ganadores	El enunciado pide bagging sobre el mejor modelo individual. Incluir el escalado dentro del estimador base garantiza que cada remuestreo se estandarice con sus propios estadísticos.
Salida probabilística	Activada	Necesaria para agregar promediando probabilidades y para poder calcular el AUC.
<code>max_samples</code>	{0,5; 0,7; 1,0}	Porcentaje de la muestra empleado en cada <i>bootstrap</i> . Submuestrear aumenta la diversidad entre modelos base, que es la fuente de la reducción de varianza.
<code>n_estimators</code>	{10, 25, 50}	En bagging, añadir estimadores nunca produce sobreajuste, sólo estabiliza; pero el coste crece linealmente (cada miembro es un SVM completo). Buscamos el punto de estabilización.
Selección de la configuración	Validación cruzada sobre <i>train</i>	Elegir mirando el test sería una fuga de selección. Se usa CV estratificada de 3 particiones por coste computacional, con doble métrica <i>accuracy</i> y AUC.

Cuadro 4.1: Decisiones de diseño del bagging y su justificación.

Se evaluaron las nueve combinaciones de la rejilla mediante validación cruzada sobre el conjunto de entrenamiento. Los resultados se recogen en el cuadro 4.2 y se representan gráficamente en la figura 4.1, empleando de nuevo *accuracy* y AUC como criterios de evaluación.

<code>max_samples</code>	Accuracy (CV)			AUC (CV)		
	$N = 10$	$N = 25$	$N = 50$	$N = 10$	$N = 25$	$N = 50$
0.5	0.8784	0.8728	0.8760	0.8829	0.8843	0.8857
0.7	0.8833	0.8792	0.8772	0.8856	0.8862	0.8862
1.0	0.8833	0.8829	0.8797	0.8856	0.8853	0.8866

Cuadro 4.2: Rendimiento del bagging en validación cruzada según el tamaño del remuestreo y el número de clasificadores agregados (N).

De estos resultados se extraen tres lecturas:

- **Efecto del número de clasificadores:** existe discrepancia en el comportamiento de las dos métricas del rendimiento, pues el *AUC* crece con los primeros modelos agregados y se **estabiliza** a partir de 25, mientras que el *accuracy* decrece y hay un descanso notable a partir de los 25 estimadores.
- **Efecto del tamaño del remuestreo:** en contra de lo que cabría esperar, submuestrear (0,5 y 0,7) tiende a **empeorar** el resultado. La explicación es que la ganancia por diversidad no llega a compensar la pérdida de información de cada modelo individual: un SVM entrenado con la mitad de las observaciones dispone de muchos menos vectores soporte para reconstruir la frontera no lineal.
- **Configuración seleccionada:** `max_samples` = 1,0 y $N = 50$, con un AUC medio en validación cruzada de 0,8866, ya que recordamos que estamos dando prioridad a esta métrica frente al *accuracy* pues tiende a beneficiar la búsqueda de clases menos representadas.

Con la configuración anterior se reentrena el ensamblado sobre la totalidad del conjunto de entrenamiento y se evalúa sobre el conjunto de test, nunca utilizado hasta este punto. Los resultados se comparan con el SVM individual en el cuadro 4.3 y en la figura 4.2. Cabe destacar que si este modelo de *Bagging* se fuese

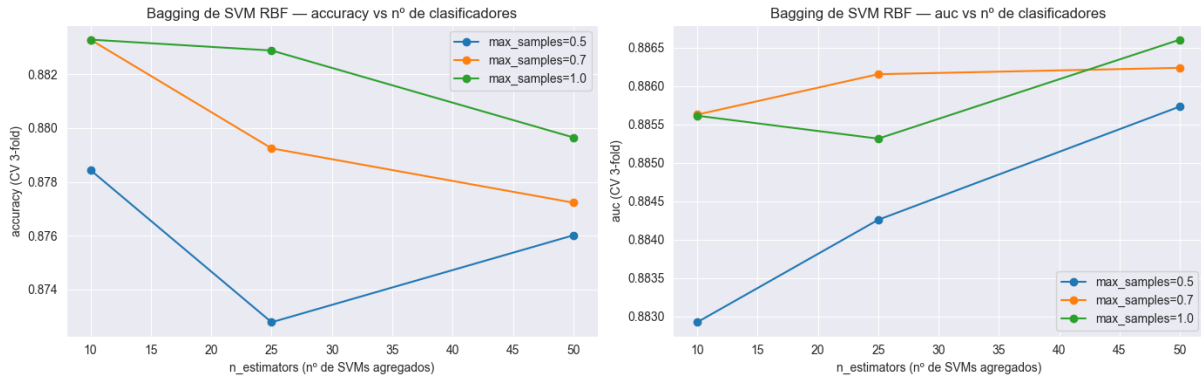


Figura 4.1: Evolución del *accuracy* y del AUC en validación cruzada frente al número de clasificadores agregados, para cada tamaño de remuestreo.

a poner en producción, se seleccionaría el modelo de 25 estimadores puesto que la ganancia en métrica no compensa el incremento en coste computacional. Sin embargo, como esto se realiza desde un punto totalmente teórico, buscamos utilizar el modelo más completo.

Métrica	SVM RBF individual	Bagging (50 SVM RBF)
Accuracy (train)	0.8937	0.9245
Accuracy (test)	0.8672	0.9096
AUC (test)	0.9211	0.9228
AUC (CV, 5 folds)	0.8923	0.8866
Recall clase 1	0.849	0.670
Falsos negativos	32	70
Falsos positivos	109	26
Gap <i>train-test</i> (accuracy)	0.0265	+0,0149

Cuadro 4.3: Comparación cuantitativa entre el mejor modelo individual y su versión ensamblada.

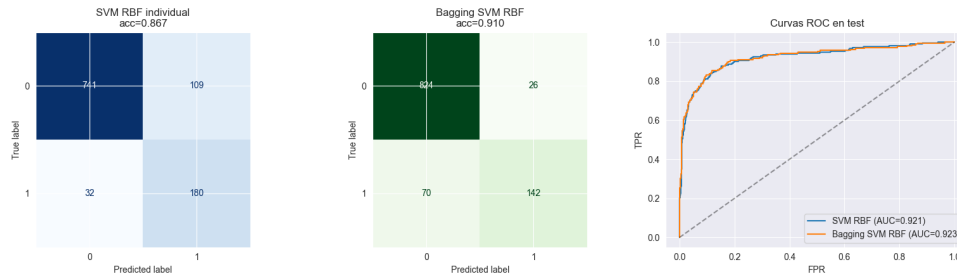


Figura 4.2: Matrices de confusión y curvas ROC en test: SVM RBF individual frente a su ensamblado por bagging.

La conclusión es un **empate técnico**: el bagging mejora marginalmente las métricas pero no es una ganancia significativa como para decantarse por un modelo concreto. De hecho, como podemos ver en la curva ROC, prácticamente son dos curvas superpuestas, demostrando que en este caso el modelo de partida es un buen modelo. Para la elección, tendríamos que estudiar la dimensión del problema y el plazo de entrega puesto para evaluar si realmente compensa emplear más tiempo por unas milésimas o centésimas de mejora.

Este resultado es **coherente con la teoría** puesto que el bagging reduce aquello que existe, y la varianza de nuestro modelo base era ya muy pequeña, es decir, tenía una buena generalización. El SVM ganador salió de una búsqueda en rejilla con validación cruzada que seleccionó explícitamente el modelo mejor regularizado, de forma que llegaba al ensamblado ajustando muy bien a los datos. El bagging brilla con

modelos base **inestables** (como el árbol de decisión), no con modelos ya estabilizados por regularización. Su aportación aquí no es precisión adicional, sino **robustez**: predicciones menos dependientes de la muestra concreta, algo valioso si el sistema se reentrena periódicamente conforme entran nuevos clientes. Sin embargo, el precio de aplicar esta técnica es computacionalmente alto ya que hay que entrenar 50 SVM en lugar de uno, con el correspondiente aumento del tiempo de entrenamiento y de inferencia.

4.2. Stacking

A diferencia del *Bagging*, en lugar de agregar réplicas de un mismo modelo mediante una regla fija (promedio o voto), combina las predicciones de **modelos base de familias distintas** mediante un **meta-modelo** que *aprende* cuál es la mejor forma de ponderarlas. El esquema es el siguiente:

1. Selección de los modelos base, buscando que cubran las carencias de unos con las fortalezas de otros.
2. Partición de los datos y entrenamiento de los modelos base sobre el conjunto de entrenamiento.
3. Generación de las predicciones de cada modelo base.
4. Construcción del **conjunto de datos meta**, en el que las predicciones de los modelos base pasan a ser las variables explicativas.
5. Entrenamiento del meta-modelo sobre ese conjunto meta.
6. Predicción final sobre observaciones nuevas.

El detalle metodológico decisivo está en el paso 3. Si las predicciones que alimentan al meta-modelo se obtuviesen sobre los mismos datos con los que se entrenaron los modelos base, el meta-modelo aprendería del sobreajuste de éstos y otorgaría un peso desmedido al modelo que mejor memoriza. Por ello las meta-características se generan mediante **validación cruzada interna** de 5 particiones: cada predicción es *out-of-fold*, es decir, procede de un modelo que no vio esa observación durante su entrenamiento. Conviene subrayar que este conjunto meta se emplea únicamente para entrenar el meta-modelo y no debe confundirse en ningún caso con el conjunto de test original, que permanece intacto.

El criterio teórico para elegir los modelos base es doble: deben tener un **sesgo bajo y similar** y estar **poco correlacionados** entre sí, esto es, generar fronteras de decisión de naturaleza distinta. Con ese criterio se seleccionaron los cuatro clasificadores del cuadro 4.4.

Antes de apilar los modelos conviene **verificar empíricamente la premisa teórica**, es decir, comprobar que existe realmente diversidad. Para ello se calcularon las métricas *out-of-fold* de cada clasificador base sobre el conjunto de entrenamiento y se estudió tanto su capacidad individual como su correlación mutua (figura 4.3):

	SVM RBF	Random Forest	Regresión logística	KNN
AUC <i>out-of-fold</i>	0.8918	0.9068	0.8104	0.8496

Clasificador	Tipo de frontera	Motivo de su inclusión y configuración
SVM RBF	No lineal suave, basada en kernel	Es el mejor modelo individual del capítulo 3 y aporta la señal principal. Se emplea con los hiperparámetros óptimos de la búsqueda en rejilla, precedido de estandarización y con salida probabilística.
Random Forest	Umbrales ortogonales sobre las variables	Captura de forma natural los efectos de umbral detectados en el EDA y las interacciones entre variables, con una lógica completamente distinta a la de un kernel. No requiere escalado. Se configura con 500 árboles.
Regresión logística	Lineal (hiperplano)	Modelo de sesgo alto pero varianza mínima: aporta la componente lineal y actúa como ancla estabilizadora del ensamblado. Precedida de estandarización.
KNN	Local, basada en distancias	Frontera puramente local, filosóficamente opuesta a las tres anteriores, lo que añade diversidad. Se fija $k = 15$, un valor moderado que suaviza la varianza propia del método.

Cuadro 4.4: Clasificadores base del stacking y justificación de su selección.

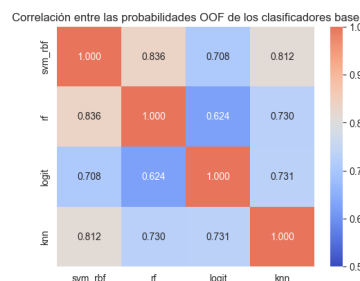


Figura 4.3: Correlación entre las probabilidades *out-of-fold* de los clasificadores base.

Ninguna pareja de modelos alcanza correlación 1, lo que confirma que estamos combinando fuentes de información distintas y no cuatro versiones del mismo modelo. Éste es justamente el escenario en el que el ensamblado puede rebajar la varianza del error: sesgos similares y correlación menor que la unidad. Notemos además que la regresión logística y el KNN son algo peores individualmente que el SVM o el Random Forest; pero esto no es un problema, ya que el stacking no exige que todos los modelos sean buenos, sino que aprenda a asignarles el peso que merecen.

4.2.1. Meta-modelo

Como meta-modelo se emplea una **regresión logística** regularizada, por tres motivos:

1. El conjunto meta tiene únicamente cuatro variables (una probabilidad por modelo base) y 3538 observaciones. Un meta-modelo complejo sobre un problema tan pequeño y tan bien separado sólo añadiría posibilidades de sobreajuste sin ganancia alguna.
2. Sus coeficientes son directamente **interpretables como los pesos** que el ensamblado asigna a cada clasificador base, de modo que el resultado es un promediado ponderado aprendido y no una caja negra adicional.
3. Al combinar **probabilidades** en lugar de etiquetas, todas las entradas del meta-modelo viven en la misma escala $[0, 1]$, lo que evita el problema de mezclar salidas de naturaleza y magnitud distintas.

El meta-modelo tiene a su vez un hiperparámetro de regularización, que se ajusta por validación cruzada sobre $C \in \{0,1; 1; 10\}$. El valor seleccionado es $C = 0,1$, es decir, la **opción más regularizada** de las tres, lo cual es coherente con un conjunto meta en el las cuatro variables tienen una correlación considerable entre sí. Con esa configuración se obtiene un AUC medio en validación cruzada de 0,9108 y un *accuracy* medio de 0,9309.

Los pesos que el meta-modelo asigna a cada clasificador base se muestran en la figura 4.4:

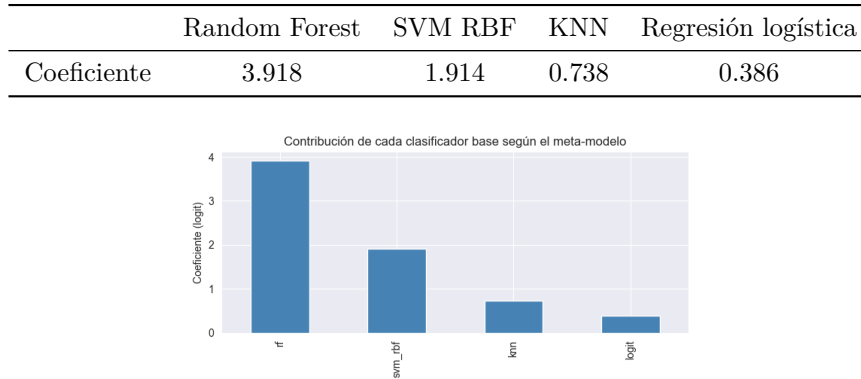


Figura 4.4: Contribución de cada clasificador base según los coeficientes del meta-modelo.

La lectura es muy ilustrativa de lo que hace realmente el stacking. El meta-modelo concentra el peso en los dos clasificadores con mejor AUC *out-of-fold* (Random Forest y SVM RBF, en ese orden) y reduce a una fracción el de los dos modelos débiles, sin llegar a anularlos. Principalmente, es el *Random Forest* el que recibe más peso, ya que al ser el que más tiende al sobreajuste, suele ser el que mejor separa. Es decir, el stacking **no descarta** a los clasificadores peores: simplemente aprende a quitarles peso, conservando la pequeña aportación diferencial que puedan tener en las regiones del espacio donde los modelos fuertes discrepan. Obsérvese también que el orden de los pesos no coincide con el orden en que los introducimos ni con nuestras expectativas previas: el Random Forest supera al SVM, algo razonable dado que el EDA había identificado efectos de umbral que un árbol captura de manera natural.

El cuadro 4.5 recoge el rendimiento en test del ensamblado y el de cada uno de sus componentes por separado.

Modelo	Accuracy (test)	AUC (test)
SVM RBF (base)	0.8672	0.9211
Random Forest (base)	0.9444	0.9383
Regresión logística (base)	0.8390	0.8487
KNN (base)	0.8484	0.8803
Stacking	0.9435	0.9386

Cuadro 4.5: Rendimiento en test del stacking frente a sus clasificadores base.

El stacking alcanza el mejor AUC de todos los modelos construidos en este trabajo (0,9386) (casi siendo igualado por el *Random Forest*) y cae unas milésimas por debajo en *accuracy*. Sobre el SVM ganador del capítulo 3, la mejora se traduce en reducir los falsos positivos de 109 a 19 y aumentar los falsos negativos de 32 a 41. Como vemos, hay una gran reducción de falsos positivos, luego aumenta claramente el *accuracy* casi una décima. La cercanía y mejora entre el AUC de validación cruzada (0,9108) y el de test (0,9386) confirma que el meta-modelo **no ha sobreajustado** el conjunto meta, que era el principal riesgo metodológico de esta técnica.

5 Conclusiones

A lo largo de este trabajo se ha construido un sistema completo de predicción del abandono de clientes, desde la depuración inicial de la base de datos hasta la comparación de cuatro alternativas de modelización. Recapitulamos los resultados obtenidos.

5.1. Comparación global de los modelos

Reunimos ahora los cuatro modelos desarrollados a lo largo del trabajo. La comparación se realiza en dos planos complementarios:

1. **Conjunto de test** (30 % de los datos, nunca utilizado en ningún ajuste): *accuracy*, AUC, curvas ROC y matrices de confusión. Es la fotografía final del rendimiento.
2. **Validación cruzada de 5 particiones sobre *train***: distribución del AUC por partición. Mide la **estabilidad** y no sólo el promedio, algo esencial aquí, porque dos modelos con la misma media pueden tener varianzas muy distintas y en producción interesa el que se comporta igual ante muestras diferentes.

Modelo	Accuracy (test)	AUC (test)	Recall clase 1
SVM lineal	0.7608	0.8493	0.8110
SVM RBF	0.8672	0.9211	0.8490
Bagging (SVM RBF)	0.9096	0.9228	0.670
Stacking	0.9435	0.9386	0.807

Cuadro 5.1: Comparativa global de los modelos desarrollados.

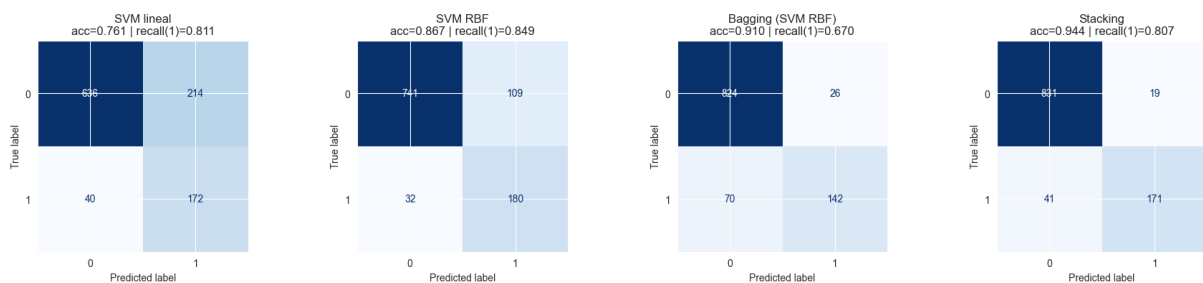
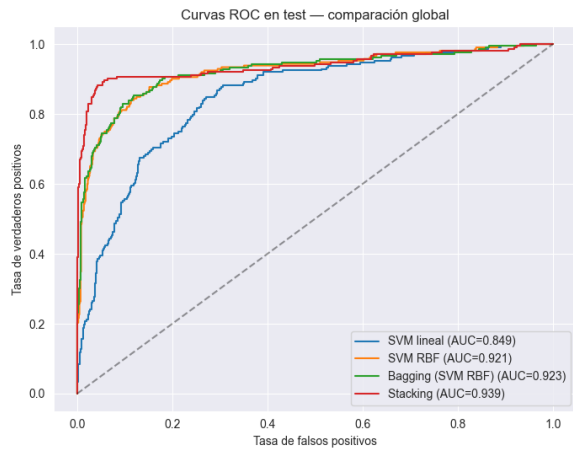
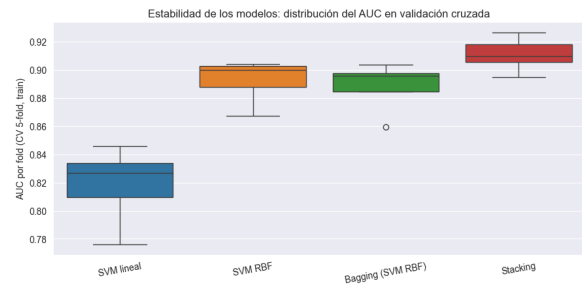


Figura 5.1: Matrices de confusión en test, con el *recall* de la clase positiva (abandono).



(a) Curvas ROC en test de los cuatro modelos.



(b) Distribución del AUC por partición en validación cruzada.

Figura 5.2: Evaluación del rendimiento y estabilidad de los modelos.

Finalmente, el cuadro 5.2 resume las ventajas e inconvenientes de cada enfoque.

Enfoque	Ventajas	Inconvenientes
SVM lineal	Muy rápido; explicabilidad muy alta; prácticamente sin riesgo de sobreajuste.	Incapaz de capturar las no linealidades detectadas en el EDA: sesgo alto y el peor rendimiento con diferencia.
SVM RBF	Captura fronteras no lineales gracias al truco del kernel; mejor modelo individual; una sola pieza sencilla de desplegar.	Dos hiperparámetros interdependientes que exigen una búsqueda cuidadosa; explicabilidad media; coste de inferencia dependiente del número de vectores soporte.
Bagging	Reduce la varianza sin tocar el sesgo; no suele empeorar al modelo base; robusto frente a reentrenamientos con muestras nuevas.	Coste computacional alto para una ganancia baja; interpretabilidad perdida.
Stacking	El mejor rendimiento global; aprende a ponderar familias diversas; el meta-modelo logístico conserva una interpretabilidad parcial mediante sus pesos.	El más complejo de construir, validar y mantener; el más caro en producción; más riesgo de sobreajuste.

Cuadro 5.2: Ventajas e inconvenientes de cada enfoque de modelización.

De la tabla y de las figuras anteriores se desprenden cuatro conclusiones:

1. **El salto relevante es el del kernel, no el del ensamblado.** La diferencia entre el SVM lineal y el SVM RBF es de casi 10 puntos en las métricas evaluadas, mientras que los ensamblados posteriores se mueven en rangos menores. Esto nos habla del dataset: el problema no era de varianza sino de **sesgo**, y quedó resuelto en el momento en que se permitió una frontera no lineal.
2. **Estabilidad.** Como vemos en la figura 5.2, los modelos son muy robustos, contando con una desviación típica del orden de las centésimas. Tanto desviaciones como medias de las métricas disminuyen en los tres modelos no lineales, viendo cómo el *bagging* gana en robustez frente al simple (lo que buscábamos con esta técnica) y el *stacking* mejora al resto de modelos.
3. **Coste y complejidad.** Las ganancias de *accuracy* se pagan con un salto notable de complejidad: el stacking exige entrenar y mantener cuatro modelos base más un meta-modelo, con validación

cruzada interna, frente a un único SVM. Al limpiar los datos y eliminar duplicados, el conjunto de datos disminuyó y se vio un incremento en la velocidad de ejecución; sin embargo, para *datasets* más grandes, serán modelos pesados de ejecutar y mantener.

Como conclusiones globales del ejercicio, podemos resumir:

Sobre el análisis exploratorio. El EDA no fue un trámite previo, sino la fase que determinó la mayor parte de las decisiones posteriores. De él salieron tres hallazgos con consecuencias directas: la redundancia perfecta entre minutos y costes, la escasa aportación de V1 frente al coste dimensional de codificarla y, sobre todo, la evidencia de **relaciones no lineales** con la variable objetivo.

Sobre el modelado con SVM. La comparación entre kernels confirmó los hallazgos del EDA y mejoramos en gran medida el modelo al introducir no linealidades. La búsqueda en rejilla mostró la **interdependencia entre C y γ** , con una diagonal de combinaciones buenas en la que un γ pequeño se compensa con un C grande y viceversa.

Sobre los métodos de ensamblado. Los dos esquemas aplicados ilustran bien lo que la teoría predice y también sus límites. El **bagging** no mejoró al SVM individual porque no había apenas varianza que reducir. El **stacking**, en cambio, sí aportó una mejora combinando familias con fronteras de decisión distintas y con una correlación entre sus predicciones suficientemente baja. El análisis de los pesos del meta-modelo mostró cómo el ensamblado reparte la confianza en cada componente y cómo el Random Forest acaba recibiendo el mayor peso por su capacidad para capturar efectos de umbral.

Modelo recomendado. Atendiendo al conjunto de criterios utilizados, la recomendación depende del uso previsto:

- Si la prioridad es el **máximo rendimiento predictivo** el modelo elegido es el **stacking**, que ofrece el mejor *accuracy* y el mejor AUC, y reduce a la mínima expresión las alertas innecesarias.
- Si se valora la **simplicidad de despliegue y mantenimiento** con un rendimiento casi idéntico, el **SVM RBF individual** es la elección: una diferencia de dos milésimas de *accuracy* difícilmente justifica multiplicar por cinco la complejidad del sistema en producción.
- El **SVM lineal** queda descartado como modelo final, si bien conserva utilidad como referencia explicable frente a la que medir el resto.

En conjunto, el trabajo confirma la idea central del módulo: los métodos de ensamblado reducen la varianza del error y casi nunca empeoran a sus modelos base, pero su ganancia es proporcional al margen de mejora que éstos dejen. Cuando el modelo individual ya está bien especificado y correctamente regularizado, como aquí ocurría con el SVM gaussiano, la aportación del ensamblado es incremental y debe ponderarse siempre frente al coste computacional y a la pérdida de interpretabilidad que conlleva.